SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

*I __**P.Sunayana(192424303)**__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

B. Optimize the code to reduce execution time for large input datasets (≥10 MB). C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures. B. Refactor the code to minimize redundant computations and memory usage. C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.

B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).

C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.

B. Optimize exponentiation operations for large prime values.

C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC)

Implementation

A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.

B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identificati on	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues

Optimizati on	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvemen t in time complexity	No improvemen t; inefficient approach retained
Analysis	20%	Demonstrate s strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrate d
Code Quality	15%	Produces clean, well structured code with proper comments and documentatio n	Code is mostly clear with minor documenta tio n gaps	Code lacks structure and proper documentati o n	Poorly written code with no documentati on

ANSWERS:

1. Debugging and Optimization of Symmetric Encryption (DES/3DES)

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

Problem: The Python program produces inconsistent ciphertext for identical inputs across multiple runs.

Possible causes and debugging:

1. **Incorrect key scheduling:** DES requires a 64-bit input key, including eight parity bits. The key schedule uses permutation choice 1 (PC-1), left rotations, and permutation choice 2 (PC-2) to generate sixteen 48-bit round keys. Incorrect rotations or permutation tables produce incorrect ciphertext.
2. **Incorrect initial and final permutations:** DES applies the Initial Permutation (IP) before encryption and the Final Permutation (FP) after the rounds. Incorrect tables or bit positions cause incorrect results.
3. **Incorrect Feistel rounds:** DES uses expansion, XOR with a round key, S-box substitution, and permutation P. Errors in any step change the ciphertext.
4. **Incorrect bit handling:** DES uses fixed-width bit operations. Missing leading zeros, incorrect indexing, or unexpected integer lengths can corrupt intermediate values.

Debugging solution: Verify all standard DES tables, test every round with known test vectors, and ensure that all intermediate values have the expected bit lengths.

B. Optimize the code for datasets of at least 10 MB.

The following methods can improve performance:

- **Use optimized cryptographic libraries:** Replace manually implemented DES/3DES operations with a tested library implementation.
- **Process data in chunks:** Read large files in fixed-size blocks rather than loading the entire dataset into memory.

Important: Chunk boundaries must respect the cipher mode and padding requirements. CBC and other stateful modes must preserve the correct chaining state between chunks.

C. Justify replacing DES/3DES with a modern cipher.

Feature	DES	3DES	AES
Security	Insecure due to its 56-bit key	Legacy algorithm with a 64-bit block size	Strong when correctly implemented

Performance	Fast in some implementations	Generally slower	Often faster with hardware acceleration
Block size	64 bits	64 bits	128 bits
Recommendation	Do not use	Avoid for new systems	Recommended

Analysis: DES is vulnerable to brute-force attacks because its key is too short. 3DES improves security over DES but is computationally expensive and has limitations associated with its 64-bit block size.

AES supports 128-, 192-, and 256-bit keys and is widely supported by modern processors and cryptographic libraries.

Conclusion: Replacing DES/3DES with AES improves security and can improve performance, especially when hardware acceleration is available. For new applications, use an authenticated mode such as AES-GCM.

---

2. Performance Analysis of Block Cipher Modes of Operation

A. Debug incorrect padding and IV handling causing decryption failures.

Problem: The Python script implements ECB, CBC, CFB, and OFB modes but experiences decryption failures and high latency.

Possible errors and solutions:

1. **Incorrect padding:** ECB and CBC generally require padding when processing data that is not a multiple of the block size. Encryption and decryption must use the same padding scheme.
2. **Incorrect IV length:** The IV must match the cipher's required block size.
3. **IV reuse:** CBC, CFB, and OFB require appropriate IV handling. Reusing an IV with the same key can reveal information or weaken security.
4. **Incorrect IV transmission:** The receiver needs the IV to decrypt the data. The IV can be transmitted alongside the ciphertext; it does not need to be secret.
5. **Incorrect padding removal:** Remove padding only after successful decryption, using the same padding scheme applied during encryption.
6. **Incorrect mode selection:** Encryption and decryption must use the same algorithm, key, mode, and required parameters.

Debugging approach: Test empty input, single-block input, multi-block input, and input lengths that are not multiples of the block size. Verify that decryption reproduces the original plaintext.

B. Refactor the code to minimize redundant computations and memory usage.

Recommended improvements:

- Use a trusted cryptographic library rather than manually implementing block cipher operations.
- Process large files in chunks to limit memory consumption.
- Avoid repeated cipher initialization when a context can safely be reused.
- Avoid unnecessary padding and unpadding operations.

For real-time systems, measure encryption time, decryption time, throughput, memory consumption, and communication overhead.

C. Compare and optimize the modes for real-time secure communication.

Mode	**Parallel Encryption**	**Padding Required**	**Main Advantage**	**Main Limitation**
ECB	Yes	Usually yes	Simple and fast	Reveals repeated patterns
CBC	No	Usually yes	Hides repeated plaintext patterns with proper IV handling	Sequential encryption and no built-in authentication
CFB	Generally sequential	No	Supports streaming-style processing	Error propagation and no built-in authentication
OFB	Keystream generation is sequential	No	Bit errors do not propagate through the keystream	IV reuse is dangerous; no built-in authentication

Analysis: ECB can be fast but is unsuitable for protecting structured data because identical plaintext blocks produce identical ciphertext blocks. CBC provides better confidentiality but

requires sequential encryption. CFB and OFB support stream-like operation without block padding, although their security depends on correct IV handling.

Decision: For a modern real-time communication system, AES-GCM is generally preferable to these four modes because it provides both confidentiality and integrity and supports efficient implementations. Where a strict requirement limits the choice to ECB, CBC, CFB, and OFB, select the mode based on the communication requirements and use a separate secure authentication mechanism.

Conclusion: AES-GCM provides an effective balance of performance and security for real-time communication when correctly implemented with unique nonces.

---

3. Debugging RSA Cryptosystem Implementation

A. Identify issues in prime generation, key generation, and modular exponentiation.

Problem: The RSA program fails when encrypting large messages and sometimes produces incorrect plaintext after decryption.

Possible errors and debugging solutions:

1. Prime generation errors

- The program may generate composite numbers instead of prime numbers.
- Small prime numbers may produce insecure RSA keys.
- The generated primes \(p\) and \(q\) must be distinct.

Solution: Use a cryptographically secure prime-generation method provided by a trusted cryptographic library.

2. Key generation errors

RSA calculates:

$$n = pq$$

$$\phi(n) = (p-1)(q-1)$$

The public exponent \(e\) must satisfy:

$$\gcd(e, \phi(n)) = 1$$

The private exponent \(d\) is calculated as:

$$d = e^{-1} \bmod \phi(n)$$

If the primes, exponent, or modular inverse are calculated incorrectly, decryption may fail.

Solution: Validate the prime numbers, ensure the public exponent is relatively prime to \(\phi(n)\), and calculate the private exponent using a reliable modular inverse function.

3. Large-message errors

RSA cannot directly encrypt arbitrarily large messages. The encoded message must fit within the RSA modulus and the selected padding scheme's size restrictions.

Solution: Use hybrid encryption: encrypt the message using a symmetric algorithm such as AES-GCM and use RSA-OAEP to encrypt the randomly generated symmetric key.

4. Modular exponentiation errors

RSA encryption and decryption use modular exponentiation:

$$C = M^e \bmod n$$

$$M = C^d \bmod n$$

Directly calculating large powers before applying the modulus is inefficient and can consume excessive memory.

Solution: Use efficient modular exponentiation, such as Python's built-in pow(base, exponent, modulus).

B. Optimize the algorithm using fast exponentiation and the Chinese Remainder Theorem.

1. Fast modular exponentiation

Fast exponentiation calculates modular powers efficiently without constructing enormous intermediate values.

Example:

ciphertext = pow(message, e, n)plaintext = pow(ciphertext, d, n)

This method performs modular exponentiation directly and is substantially more efficient than calculating the complete power first.

2. Chinese Remainder Theorem (CRT)

CRT accelerates RSA private-key operations by calculating the result modulo \(p\) and modulo \(q\), then recombining the results.

$$d_P = d \bmod (p-1)$$

$$d_Q = d \bmod (q-1)$$

$$q_{\text{inv}} = q^{-1} \bmod p$$

$$m_1 = C^{d_P} \bmod p$$

$$m_2 = C^{d_Q} \bmod q$$

Advantages of CRT:

- Reduces the computational cost of private-key operations.
- Improves RSA decryption and signing performance.
- Makes large-key cryptographic operations more efficient.

Security consideration: CRT implementations must protect against fault attacks and side-channel leakage. Using a vetted cryptographic library is safer than implementing RSA manually.

C. Evaluate the trade-offs between key size, security, and computational performance.

RSA Key Size	Security	Performance	Suitability
1024 bits	Inadequate for modern security	Faster	Not recommended
2048 bits	Common modern baseline	Moderate	Suitable for many applications
3072 bits	Stronger security margin	Slower	Appropriate when stronger protection is required
4096 bits	Higher security margin	Generally slower	Used when requirements justify the additional cost

Analysis: Larger RSA keys increase the difficulty of factoring the modulus but require more computation for key generation and cryptographic operations. Smaller keys provide better speed but weaker security.

For modern applications, RSA-2048 is a common minimum baseline, while RSA-3072 may be selected for a longer security horizon.

Conclusion: RSA performance can be improved through fast modular exponentiation and CRT. Hybrid encryption solves the large-message limitation, while appropriate key sizes and secure padding protect confidentiality. A trusted cryptographic library should handle key generation, encryption, and decryption.

---

4. Optimization and Security Analysis of Diffie–Hellman Key Exchange

A. Debug incorrect shared-key generation due to improper modular arithmetic.

Problem: The Diffie–Hellman implementation generates incorrect shared keys because of errors in modular arithmetic.

$$A = g^a \bmod p$$

$$B = g^b \bmod p$$

$$K_A = B^a \bmod p$$

$$K_B = A^b \bmod p$$

$$K_A = K_B = g^{ab} \bmod p$$

Possible errors:

1. Applying the modulus incorrectly.
2. Using different values of \(p\) or \(g\).
3. Accidentally exchanging private values instead of public values.
4. Using insecure or improperly generated private values.

Debugging solution: Use modular exponentiation, verify that both users use the same approved parameters, validate public values, and test that both independently calculated shared secrets are equal.

B. Optimize exponentiation operations for large prime values.

The main optimization is to use efficient modular exponentiation rather than calculating a complete power.

Python implementation:

```
import secrets
p = 23
g = 5
a = secrets.randbelow(p - 3) + 2
b = secrets.randbelow(p - 3) + 2
```

```
A = pow(g, a, p)
B = pow(g, b, p)
key_a = pow(B, a, p)
key_b = pow(A, b, p)
print("User A public key:", A)
print("User B public key:", B)
print("Shared keys match:", key_a == key_b)
```

Sample output:

```
User A public key: 10
User B public key: 17
Shared keys match: True
```

The public keys and private values can vary between executions.

Optimization benefits:

- pow(base, exponent, modulus) performs efficient modular exponentiation.
- secrets generates cryptographically suitable random values.

Important: This example uses small demonstration parameters and is not secure for real communication. Production systems must use standardized, sufficiently large parameters and an approved key-exchange protocol.

C. Mitigate man-in-the-middle attacks and evaluate the performance impact.

Security threat: Man-in-the-Middle (MITM) attack

An attacker may intercept the public keys exchanged by the users and substitute their own public values. The attacker can then establish separate shared secrets with both parties and potentially read or modify their communication.

Mitigation techniques:

1. **Digital signatures:** Sign the key-exchange messages to verify their authenticity.
2. **Certificates:** Use trusted certificates to authenticate the identities associated with public keys.
3. **Public-key verification:** Verify signatures and certificate chains before accepting exchanged values.
4. **Public-value validation:** Reject invalid or out-of-range public values.
5. **Key derivation:** Apply a suitable key derivation function, such as HKDF, to the shared secret.

Performance analysis:

Technique	Security Benefit	Performance Impact
Efficient modular exponentiation	Correct and efficient key calculation	Reduces computation time
Digital signatures	Authenticates key-exchange messages	Adds signing and verification operations
Certificate validation	Verifies identity and trust	Adds validation overhead
Public-value validation	Rejects invalid inputs	Usually adds relatively little overhead
Key derivation function	Derives suitable cryptographic session keys	Adds a small amount of computation

Conclusion: Efficient modular exponentiation improves Diffie–Hellman performance. Adding authentication and public-value validation protects against important attacks, although authentication introduces some additional computation and communication .

---

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC)

A. Identify and fix errors in point addition and scalar multiplication.

Problem: The ECC program produces incorrect results for certain curve parameters.

ECC commonly operates over a finite field using a curve of the form:

$$y^2 = x^3 + ax + b \pmod{p}$$

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}$$

Possible errors:

1. Incorrect point addition

$P = (x_1, y_1)$ and $Q = (x_2, y_2)$, where $x_1 \neq x_2$, calculate:

$$\lambda = (y_2 - y_1)(x_2 - x_1)^{-1} \bmod p$$

$$x_3 = (\lambda^2 - x_1 - x_2) \bmod p$$

$$y_3 = (\lambda(x_1 - x_3) - y_1) \bmod p$$

2. Incorrect point doubling

When \(P=Q\), the slope must be calculated using:

$$\lambda = (3x_1^2 + a)(2y_1)^{-1} \bmod p$$

Using the ordinary point-addition formula for point doubling produces incorrect results.

3. Point at infinity

The point at infinity acts as the identity element:

$$P + \mathcal{O} = P$$

Failure to handle this special point can break scalar multiplication.

4. Inverse points

$$P + Q = \mathcal{O}$$

The implementation must handle this case instead of attempting an invalid modular inverse.

5. Invalid curve parameters

Points must satisfy the curve equation, and the curve parameters must define a valid nonsingular curve.

Debugging solution: Check the curve equation, validate input points, implement modular inverses correctly, handle point doubling and inverse points separately, and test the group-operation properties.

B. Optimize the implementation using double-and-add.

Repeated addition requires approximately \(k\) point additions. The double-and-add method reduces the number of operations to a quantity proportional to the number of bits in \(k\).

Algorithm:

1. Initialize the result to the point at infinity.
2. Set the current point to \(P\).
3. Examine the binary representation of \(k\).
4. If the current bit is 1, add the current point to the result.
5. Double the current point.
6. Continue until all bits have been processed.
7. Return the result.

Python-style pseudocode:

```
function scalar_multiply(k, P):
    result = POINT_AT_INFINITY
```

```
    current = P
    while k > 0:
        if k is odd:
            result = point_add(result, current)
 current = point_add(current, current)
k = k // 2
    return result
```

Optimization benefits:

- Reduces the number of point operations.
- Improves performance for large scalar values.
- Reduces unnecessary repeated additions.
- Makes the algorithm more practical for key exchange and digital signatures.

Security consideration: A straightforward double-and-add implementation may leak information through timing or other side channels. Production cryptography should use constant-time operations and vetted ECC libraries.

C. Compare optimized ECC with RSA in terms of efficiency and security.

Criterion	**RSA**	**ECC**
Mathematical foundation	Integer factorization	Elliptic-curve discrete logarithm problem
Key size	Relatively large	Smaller for comparable security
Key generation	Can be computationally expensive	Often efficient
Private-key operations	Can be expensive, especially with large keys	Often efficient with optimized implementations

Security-strength comparison:

For approximately comparable classical security:

- RSA-2048 provides roughly 112 bits of security.
- ECC using a well-selected 256-bit curve provides roughly 128 bits of security.

Analysis: ECC provides strong security with substantially smaller keys. This reduces storage requirements and the amount of data exchanged, making ECC particularly suitable for mobile devices, embedded systems, and IoT devices.

Conclusion: ECC with double-and-add scalar multiplication offers an efficient approach to elliptic-curve operations. Its smaller keys and lower communication overhead make it generally preferable for resource-constrained environments. Secure implementations should use standardized curves, validated parameters, constant-time operations, and established cryptographic libraries.